32-performance

32. Web Performance Optimization

Level: Intermediate
Jam: 8 (4 minggu × 2 sesi)
Prasyarat: HTML, CSS, JavaScript, TypeScript dasar, React/Vue dasar
Output: Aplikasi web dengan skor Lighthouse ≥ 90 di semua kategori

Tujuan Pembelajaran

Setelah modul ini, kamu bisa: - Ukur & optimasi Core Web Vitals (LCP, FID, CLS, INP) - Terapkan lazy loading gambar & komponen - Manfaatkan browser caching & Service Worker - Analisis & optimasi bundle size - Integrasi Lighthouse CI di pipeline - Pantau performa web di production

Materi

Sesi	Topik	File
1	Web Vitals — LCP, FID, CLS, INP, PerformanceObserver	01-web-vitals.md
2	Lazy Loading, Code Splitting & Caching	02-lazy-loading-caching.md
3	Bundle Optimization — Tree Shaking, Image Opt, Font Loading	03-bundle-optimization.md
4	Lighthouse CI, Performance Budgets & Monitoring	04-lighthouse-ci.md

Output Akhir Modul

> **Performance Dashboard App** — aplikasi yang mendemonstrasikan teknik optimasi performa dan mencapai skor Lighthouse ≥ 90 di semua kategori (Performance, Accessibility, SEO, Best Practices).

AI Prompt Exercises

Sepanjang modul, latihan pake AI: - “Explain this PerformanceObserver code line by line” - “Find the bottleneck in this bundle analysis report” - “Optimize this image loading strategy for Core Web Vitals” - “Generate a lighthouse-ci.yml for this project” - “Refactor this component to use lazy loading” - “Create a caching strategy for this API endpoint”

32.1 Web Vitals — LCP, FID, CLS, INP

Apa itu Web Vitals?

Web Vitals adalah metrik resmi Google buat ngukur **pengalaman pengguna** di web. Ada 4 metrik inti:

Metrik	Nama	Ngukur Apa	Target
LCP	Largest Contentful Paint	Waktu render elemen konten terbesar	≤ 2.5 detik
FID	First Input Delay	Waktu respon ke interaksi pertama	≤ 100 ms
CLS	Cumulative Layout Shift	Stabilitas visual / layout shift	≤ 0.1
INP	Interaction to Next Paint	Responsivitas interaksi keseluruhan	≤ 200 ms

---

1. LCP (Largest Contentful Paint)

Penjelasan

LCP ngukur kapan **elemen konten terbesar** di viewport selesai dirender. Elemen yang diukur: `<img>`, `<svg>`, `<video>`, element dengan

`background-image`, atau text block besar.

Cara Ukur

```
// PerformanceObserver untuk LCP
new PerformanceObserver((list) => {
  const entries = list.getEntries();
  const lastEntry = entries[entries.length - 1];
  console.log('LCP:', lastEntry.startTime / 1000, 'detik');
  console.log('Elemen:', (lastEntry as any).element);
}).observe({ type: 'largest-contentful-paint', buffered: true });
```

Cara Fix

Masalah	Solusi
Gambar lambat load	Optimasi ukuran gambar, pake next-gen format (WebP/AVIF)
Font besar blocking render	`font-display: swap` atau preload font
Hero image butuh request besar	Gunakan `<img fetchpriority="high">`
Server slow TTFB	Cache CDN, optimasi server, gunakan SSG

```
<!-- Preload hero image -->
<link rel="preload" href="hero.webp" as="image" fetchpriority="high">

<!-- font-display: swap -->
<style>
@font-face {
  font-family: 'Inter';
  src: url('/fonts/Inter.woff2') format('woff2');
  font-display: swap;
}
</style>
```

---

2. FID (First Input Delay)

Penjelasan

FID ngukur **delay** antara user pertama kali interaksi (klik, tap, key-down) sampe browser bisa nge-handle event handler-nya. Penyebab utama: **main thread diblokir** oleh JavaScript berat yang lagi jalan.

Cara Ukur

```
// PerformanceObserver untuk FID
new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    const fid = entry as PerformanceEventTiming;
    console.log('FID delay:', fid.processingStart - fid.startTime, 'ms');
    console.log('Event type:', fid.name);
  }
}).observe({ type: 'first-input', buffered: true });
```

Cara Fix

Masalah	Solusi
JavaScript besar blocking main thread	Code splitting, lazy load JS
Long task (>50ms)	Break jadi microtask, pake `requestIdleCallback`
Third-party scripts blocking	`async` / `defer`, load setelah interaksi
Hydration berat (framework)	Partial hydration, island architecture

```
// requestIdleCallback — jalankan task saat browser senggang
function processHeavyTask(data: number[]) {
  const chunkSize = 50;
  let index = 0;

  function processChunk() {
    const end = Math.min(index + chunkSize, data.length);
    for (let i = index; i < end; i++) {
      // Lakukan task berat
      data[i] = data[i] * 2;
    }
    index = end;

    if (index < data.length) {
      requestIdleCallback(processChunk, { timeout: 2000 });
    }
  }

  requestIdleCallback(processChunk, { timeout: 2000 });
}
```

3. CLS (Cumulative Layout Shift)

Penjelasan

CLS ngukur **seberapa banyak elemen bergerak** saat halaman lagi dimuat. Layout shift terjadi kalo elemen udah di-render tapi ukurannya berubah pas asset lain (gambar, iklan, font) selesai dimuat.

Cara Ukur

```
// PerformanceObserver untuk CLS
let clsValue = 0;

new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    if (!(entry as any).hadRecentInput) {
      clsValue += (entry as any).value;
    }
  }
  console.log('CLS:', clsValue);
}).observe({ type: 'layout-shift', buffered: true });
```

Cara Fix

Masalah	Solusi
Gambar tanpa dimensi	Selalu set `width` + `height` di `<img>`
Iklan / embed dinamis	Reserve space dengan CSS `min-height`
Font loading	`font-display: optional` + `size-adjust`
Dynamic content injected di atas konten	Inject di bawah viewport atau pake skeleton

```
<!-- Set dimensi gambar untuk cegah CLS -->
<img
  src="photo.webp"
  width="800"
  height="600"
  alt="Deskripsi"
  loading="lazy"
  style="aspect-ratio: 800 / 600"
>
```

```html
<!-- Reserve space untuk iklan -->
<div style="min-height: 250px; width: 100%;">
  <!-- Iklan dimuat di sini -->
</div>
```

---

4. INP (Interaction to Next Paint)

Penjelasan

INP adalah metrik baru (2024) yang ngukur **responsivitas keseluruhan** interaksi user — bukan cuma yang pertama (FID). Ngukur delay dari interaksi sampe frame berikutnya tampil.

Cara Ukur

```
// PerformanceObserver untuk INP
new PerformanceObserver((list) => {
  const entries = list.getEntries();
  const interactions = entries.filter(
    (e): e is PerformanceEventTiming => e.entryType === 'event'
  );

  for (const entry of interactions) {
    const delay = entry.processingStart - entry.startTime;
    const duration = entry.duration;
    console.log(`INP — ${entry.name}: delay=${delay}ms, duration=${duration}ms`)
  }
}).observe({ type: 'event', buffered: true, durationThreshold: 16 });
```

Cara Fix

Masalah	Solusi
Event handler terlalu berat	Pindahin ke Web Worker atau `setTimeout`
Re-render besar setelah interaksi	Virtual list, memoization, `content-visibility`
Third-party scripts blocking	Defer atau load async
Rendering lambat	`will-change` CSS, GPU acceleration

```
/* content-visibility — skip render elemen di luar viewport */
.lazy-section {
```

```
  content-visibility: auto;
  contain-intrinsic-size: 0 500px;
}

// Debounce input handler biar ga render tiap keystroke
function debounce<T extends (...args: unknown[]) => void>(
  fn: T, ms: number
): (...args: Parameters<T>) => void {
  let timer: ReturnType<typeof setTimeout>;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), ms);
  };
}

const handleSearch = debounce(async (query: string) => {
  const results = await fetch(`/api/search?q=${query}`);
  // update UI
}, 300);
```

Report ke Analytics

Kirim metrik ke backend buat monitoring:

```
function sendToAnalytics(metrics: {
  name: string;
  value: number;
  rating: 'good' | 'needs-improvement' | 'poor';
}) {
  const body = JSON.stringify({
    metric: metrics.name,
    value: metrics.value,
    rating: metrics.rating,
    url: window.location.pathname,
    userAgent: navigator.userAgent,
  });

  // Kirim pake sendBeacon — ga blocking, reliable walau page unload
  navigator.sendBeacon('/api/metrics', body);
}

// Observer semua metrik
function observeWebVitals() {
  const lcpObserver = new PerformanceObserver((list) => {
    const entries = list.getEntries();
```

```
    const lcp = entries[entries.length - 1];
    sendToAnalytics({
      name: 'LCP',
      value: lcp.startTime,
      rating: lcp.startTime <= 2500 ? 'good' : lcp.startTime <= 4000 ? 'needs-im
    });
  });
  lcpObserver.observe({ type: 'largest-contentful-paint', buffered: true });

  // Observer untuk CLS
  let clsValue = 0;
  const clsObserver = new PerformanceObserver((list) => {
    for (const entry of list.getEntries()) {
      if (!(entry as any).hadRecentInput) {
        clsValue += (entry as any).value;
      }
    }
  });
  clsObserver.observe({ type: 'layout-shift', buffered: true });

  // Kirim CLS saat page unload
  window.addEventListener('beforeunload', () => {
    sendToAnalytics({
      name: 'CLS',
      value: clsValue,
      rating: clsValue <= 0.1 ? 'good' : clsValue <= 0.25 ? 'needs-improvement'
    });
  });
}

observeWebVitals();
```

Tabel Ringkasan Metrik

Metrik	Target Baik	Tool Ukur	Cara Fix Utama
LCP	≤ 2500ms	PerformanceObserver, Lighthouse	Optimasi gambar, preload hero, CDN
FID	≤ 100ms	PerformanceObserver, web-vitals	Code splitting, kurangi JS blocking
CLS	≤ 0.1	PerformanceObserver, Lighthouse	Set dimensi gambar, reserve space

Metrik	Target Baik	Tool Ukur	Cara Fix Utama
INP	≤ 200ms	PerformanceObserver, web-vitals	Debounce, delegasi event, worker

---

Latihan

1. **LCP Reporter** — tulis fungsi `reportLCP()` yang ngukur LCP dan kirim ke console. Pakai `PerformanceObserver`. Tambahin threshold warn (kuning >2500ms, merah >4000ms)
2. **CLS Debugger** — bikin halaman HTML dengan 3 gambar tanpa `width`/`height`. Ukur CLS pake `PerformanceObserver`. Terus tambahin dimensi — bandingin nilai CLS sebelum-sesudah
3. **INP Simulator** — bikin fungsi berat yang blocking main thread selama 500ms (loop besar). Ukur INP pake `PerformanceObserver`. Terus refactor pake `requestIdleCallback`:
4. **Analytics Logger** — gabungin LCP + CLS + INP observer jadi satu fungsi `observeAndReport()`. Kirim data ke console.table(). Format: name | value | rating | timestamp

32.2 Lazy Loading, Code Splitting & Caching

---

1. Image Lazy Loading

Native Lazy Loading (`loading=lazy`)

Cara termudah — browser handle sendiri:

```
<img
  src="photo.webp"
  loading="lazy"
  width="800"
  height="600"
  alt="Deskripsi"
>

<!-- loading="eager" — load langsung (default buat hero images) -->
<img src="hero.webp" loading="eager" fetchpriority="high" alt="Hero">
```

Nilai	Perilaku
`lazy`	Muat gambar pas mendekati viewport (default 1250px)
`eager`	Muat gambar segera, ga peduli posisi
`auto`	Default browser — biasanya eager

IntersectionObserver — Kontrol Manual

Native `loading=lazy` ga selalu cukup (browser support, jarak trigger).
Pake `IntersectionObserver` buat kontrol lebih:

```
function lazyLoadImages() {
  const images = document.querySelectorAll<HTMLImageElement>('img[data-src]');

  const observer = new IntersectionObserver(
    (entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) {
          const img = entry.target as HTMLImageElement;
          img.src = img.dataset.src!;
          img.removeAttribute('data-src');
          observer.unobserve(img);

          // Fade in efek
          img.style.opacity = '1';
          img.style.transition = 'opacity 0.3s ease-in';
        }
      });
    },
    {
      rootMargin: '200px', // Mulai load 200px sebelum masuk viewport
      threshold: 0.01,
    }
  );

  images.forEach((img) => {
    img.style.opacity = '0';
    observer.observe(img);
  });
}

document.addEventListener('DOMContentLoaded', lazyLoadImages);
```

```
<img
  data-src="real-image.webp"
  src="placeholder-lowres.webp"
```

```
  width="800"
  height="600"
  alt="Lazy loaded"
>
```

Priority Hints

```html
<!-- High priority — hero image harus cepat -->
<img src="hero.webp" fetchpriority="high" alt="Hero">

<!-- Low priority — gambar di bawah fold -->
<img src="footer.webp" fetchpriority="low" loading="lazy" alt="Footer">
```

---

2. Code Splitting

Dynamic Import — JavaScript

```javascript
// Before — semua di satu bundle
import { heavyFunction } from './heavy-module';

// After — baru load kalo dipanggil
async function loadFeature() {
  const { heavyFunction } = await import('./heavy-module');
  heavyFunction();
}

// Dynamic import dengan error handling
async function loadChart() {
  try {
    const ChartModule = await import('./ChartComponent');
    const chart = new ChartModule.default();
    chart.render();
  } catch (err) {
    console.error('Gagal load chart module:', err);
    renderFallbackChart();
  }
}
```

React.lazy — Lazy Load Component

```javascript
import { lazy, Suspense } from 'react';
import LoadingSpinner from './LoadingSpinner';

// Lazy load — baru di-download kalo di-render
```

```tsx
const Dashboard = lazy(() => import('./Dashboard'));
const AnalyticsChart = lazy(() => import('./AnalyticsChart'));
const UserSettings = lazy(() => import('./UserSettings'));

function App() {
  const [page, setPage] = useState<'dashboard' | 'analytics' | 'settings'>('dash

  return (
    <div>
      <nav>
        <button onClick={() => setPage('dashboard')}>Dashboard</button>
        <button onClick={() => setPage('analytics')}>Analytics</button>
        <button onClick={() => setPage('settings')}>Settings</button>
      </nav>

      <Suspense fallback={<LoadingSpinner />}>
        {page === 'dashboard' && <Dashboard />}
        {page === 'analytics' && <AnalyticsChart />}
        {page === 'settings' && <UserSettings />}
      </Suspense>
    </div>
  );
}
```

Route-based Splitting (React Router)

```tsx
import { lazy, Suspense } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';

const Home = lazy(() => import('./pages/Home'));
const ProductList = lazy(() => import('./pages/ProductList'));
const ProductDetail = lazy(() => import('./pages/ProductDetail'));
const Checkout = lazy(() => import('./pages/Checkout'));
const NotFound = lazy(() => import('./pages/NotFound'));

function App() {
  return (
    <BrowserRouter>
      <Suspense fallback={<div>Loading...</div>}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/produk" element={<ProductList />} />
          <Route path="/produk/:id" element={<ProductDetail />} />
          <Route path="/checkout" element={<Checkout />} />
          <Route path="*" element={<NotFound />} />
```

```
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```

Preload / Prefetch Strategis

```
<!-- preload — butuh SEGERA buat halaman ini -->
<link rel="preload" href="/fonts/Inter-Bold.woff2" as="font" crossorigin>

<!-- prefetch — mungkin dibutuhkan di navigasi berikutnya -->
<link rel="prefetch" href="/js/product-detail.chunk.js" as="script">

<!-- preconnect — siapkan koneksi ke origin lain -->
<link rel="preconnect" href="https://api.example.com">
```

---

3. Browser Caching

Cache-Control — HTTP Header Paling Penting

Direktif	Maksud	Contoh
`max-age=31536000`	Cache 1 tahun	File statis: CSS, JS, gambar
`no-cache`	Validasi tiap request	HTML, API response
`no-store`	Jangan simpen sama sekali	Data sensitif
`must-revalidate`	Wajib cek ulang kalo expired	

```
# Nginx config — cache file statis
location /assets/ {
  expires 1y;
  add_header Cache-Control "public, immutable, max-age=31536000";
}

location /api/ {
  add_header Cache-Control "no-cache, private";
}
```

```
// Express — setting Cache-Control di response
import express from 'express';

const app = express();

app.use('/static', express.static('public', {
```

```
  maxAge: '1y',
  immutable: true,
  setHeaders: (res) => {
    res.setHeader('Cache-Control', 'public, immutable, max-age=31536000');
  },
}));

app.get('/api/user', (req, res) => {
  res.setHeader('Cache-Control', 'no-cache');
  res.json({ id: 1, name: 'Budi' });
});
```

ETag — Validasi Cache

ETag adalah hash konten. Browser kirim `If-None-Match`, server balas `304 Not Modified` kalo konten sama.

```
import crypto from 'node:crypto';
import express from 'express';

const app = express();

app.get('/api/products', async (req, res) => {
  const products = await getProducts();
  const hash = crypto.createHash('md5').update(JSON.stringify(products)).digest(

  // Set ETag
  res.setHeader('ETag', `"${hash}"`);

  // Cek kalo browser kirim ETag yang sama
  if (req.headers['if-none-match'] === `"${hash}"`) {
    res.status(304).end(); // Not modified — ga kirim body
    return;
  }

  res.json(products);
});
```

Ringkasan Strategi Cache

Tipe Resource	Cache-Control	ETag	Alasan
`*.css`, `*.js`	`max-age=31536000, immutable`	Opsional	File di-hash by bundler

Tipe Resource	Cache-Control	ETag	Alasan
Gambar statis	`max-age=31536000, public`	Opsional	Jarang berubah
API response	`no-cache` atau `max-age=60`	Wajib	Data bisa berubah
HTML	`no-cache`	Wajib	Konten dinamis
Data sensitif	`no-store`	—	Jangan di-cache

---

4. Service Worker — Cache Strategies

Register Service Worker

```ts
// main.ts — daftarin service worker
if ('serviceWorker' in navigator) {
  window.addEventListener('load', async () => {
    try {
      const registration = await navigator.serviceWorker.register('/sw.ts', {
        scope: '/',
      });
      console.log('SW registered:', registration.scope);
    } catch (err) {
      console.error('SW registration failed:', err);
    }
  });
}
```

Stale-While-Revalidate

Kirim data dari cache dulu (cepat), update cache dari network di background.

```ts
// sw.ts
const CACHE_NAME = 'v1-stale-while-revalidate';

self.addEventListener('fetch', (event: FetchEvent) => {
  if (event.request.url.includes('/api/')) {
    event.respondWith(staleWhileRevalidate(event.request));
  }
});

async function staleWhileRevalidate(request: Request): Promise<Response> {
```

```
  const cache = await caches.open(CACHE_NAME);
  const cachedResponse = await cache.match(request);

  // Fetch dari network (di background)
  const fetchPromise = fetch(request).then(async (networkResponse) => {
    if (networkResponse.ok) {
      await cache.put(request, networkResponse.clone());
    }
    return networkResponse;
  });

  // Balas dari cache dulu (kalo ada)
  return cachedResponse || fetchPromise;
}
```

Cache-First

Cari di cache dulu. Kalo ga ada, ambil dari network dan simpan.

```
// sw.ts
self.addEventListener('fetch', (event: FetchEvent) => {
  if (event.request.url.match(/\.(png|jpg|webp|svg|css|js)$/)) {
    event.respondWith(cacheFirst(event.request));
  }
});

async function cacheFirst(request: Request): Promise<Response> {
  const cache = await caches.open('v1-cache-first');
  const cached = await cache.match(request);

  if (cached) {
    return cached;
  }

  const networkResponse = await fetch(request);
  if (networkResponse.ok) {
    cache.put(request, networkResponse.clone());
  }
  return networkResponse;
}
```

Network-First (API)

Coba network dulu. Kalo gagal (offline), ambil dari cache.

```ts
// sw.ts
self.addEventListener('fetch', (event: FetchEvent) => {
  if (event.request.url.includes('/api/')) {
    event.respondWith(networkFirst(event.request));
  }
});

async function networkFirst(request: Request): Promise<Response> {
  const cache = await caches.open('v1-api');

  try {
    const networkResponse = await fetch(request);
    if (networkResponse.ok) {
      cache.put(request, networkResponse.clone());
    }
    return networkResponse;
  } catch (err) {
    const cached = await cache.match(request);
    if (cached) {
      return cached;
    }
    // Fallback — balik ke offline page
    return caches.match('/offline.html');
  }
}
```

Cache-Only (Asset Statis)

Cuma dari cache, ga pake network. Cocok buat icon, logo, font.

```ts
// sw.ts — install event: pre-cache file statis
const PRECACHE_ASSETS = [
  '/',
  '/index.html',
  '/styles/main.css',
  '/scripts/main.js',
  '/images/logo.svg',
  '/fonts/Inter.woff2',
  '/offline.html',
];

self.addEventListener('install', (event) => {
  event.waitUntil(
    caches.open('v1-static').then((cache) => cache.addAll(PRECACHE_ASSETS))
  );
});
```

```
self.addEventListener('fetch', (event: FetchEvent) => {
  if (PRECACHE_ASSETS.includes(new URL(event.request.url).pathname)) {
    event.respondWith(caches.match(event.request));
  }
});
```

Perbandingan Strategi

Strategy	Kecepatan	Freshness	Cocok Buat
Cache-First	⚡ Tercepat	□ Bisa basi	Asset statis (CSS, JS, gambar)
Network-First	□ Lambat (kalo online)	□ Tersegar	API, HTML dinamis
Stale-While-Revalidate	⚡ Cepat (cache)	□ Update background	API yang ga realtime, halaman
Cache-Only	⚡ Tercepat	□ Tergantung pre-cache	Asset yang jarang berubah
Network-Only	□ Normal	□ Tersegar	Data sensitif, realtime

Latihan

1. **Lazy Gallery** — bikin halaman dengan 20 gambar, masing-masing pake `data-src`. Implementasi IntersectionObserver buat load gambar pas masuk viewport. Tambahin loading placeholder (blur/lowres)
2. **Route Splitter** — bikin React app mini 3 halaman (Home, Products, About). Pake `React.lazy` + `Suspense` buat code splitting. Ukur beda bundle size sebelum-sesudah pake Network tab
3. **SW Cache Dashboard** — bikin service worker dengan strategi: cache-first buat CSS/JS/font, network-first buat API `/api/users`, stale-while-revalidate buat halaman HTML. Test pake offline mode DevTools
4. **ETag API** — bikin Express endpoint `/api/products` yang:
   - Generate ETag dari JSON.stringify + md5
   - Check `if-none-match` header

- Balas 304 kalo sama
- Test pake curl dengan `If-None-Match`

32.3 Bundle Optimization — Tree Shaking, Image Opt, Font Loading

---

1. Bundle Analysis

Webpack Bundle Analyzer

Lihat isi bundle — siapa paling gede, mana yang bisa di-split.

```
npm install --save-dev webpack-bundle-analyzer
```

```
// webpack.config.js
const BundleAnalyzerPlugin = require('webpack-bundle-analyzer').BundleAnalyzerPl

module.exports = {
  plugins: [
    new BundleAnalyzerPlugin({
      analyzerMode: 'static',      // Hasilkan HTML report
      reportFilename: 'report.html',
      openAnalyzer: false,
      generateStatsFile: true,
      statsFilename: 'stats.json',
    }),
  ],
};
```

Vite Bundle Visualizer

```
npm install --save-dev rollup-plugin-visualizer
```

```
// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { visualizer } from 'rollup-plugin-visualizer';

export default defineConfig({
  plugins: [
    react(),
    visualizer({
      filename: 'dist/stats.html',
```

```
      open: false,
      gzipSize: true,
      brotliSize: true,
    }),
  ],
});
```

Buka `dist/stats.html` — liat treemap module. Hijau = aman, Merah = masalah.

Analisis Manual

```
# Lihat file size di dist
ls -lh dist/assets/
du -sh dist/

# Webpack stats JSON analyzer
npx webpack-bundle-analyzer dist/stats.json

# Lighthouse report — liat "Reduce JavaScript execution time"
```

Common Bottlenecks

Masalah	Tanda	Solusi
Lodash / moment.js utuh	Bundle > 200 KB	Import spesifik: `import debounce from 'lodash/debounce'`
Library chart besar	Chart.js > 200 KB	Pake lightweight: uPlot, Chart.js tree-shake
Dua versi React	Duplikat module	`resolve.alias` / dedupe
CSS library penuh	Tailwind purged?	`purge: true`

---

2. Tree Shaking

Tree shaking = **buang kode yang ga dipake**. Webpack/Vite otomatis kalo pake ES Module.

Cara Kerja

```ts
// utils.ts — export banyak fungsi
export function formatDate(date: Date): string { /* ... */ }
export function formatCurrency(amount: number): string { /* ... */ }
export function generateSlug(text: string): string { /* ... */ }
export function debounce<T>(fn: T, ms: number): T { /* ... */ }
export function throttle<T>(fn: T, ms: number): T { /* ... */ }

// app.ts — cuma pake 2 dari 5 fungsi
// Tree shaking: formatDate & debounce DO simpan, 3 sisanya DO buang
import { formatDate, debounce } from './utils';
```

Aturan Biar Tree Shaking Berfungsi

Aturan	Penjelasan
Pake **ES Module** (`import`/`export`)	CommonJS (`require`) GA bisa di-tree-shake
Side-effect free	`package.json` → `"sideEffects": false`
Jangan import semuanya	`import * as _ from 'lodash'` → semua ikut
Import spesifik	`import { debounce } from 'lodash-es'`

Config Tree Shaking

```ts
// package.json — bilang ke bundler "ga ada side effects"
{
  "sideEffects": [
    "*.css",
    "*.scss"
  ]
}
```

```ts
// webpack.config.js
module.exports = {
  mode: 'production', // Tree shaking otomatis di production
  optimization: {
    usedExports: true,
    sideEffects: true,
  },
};
```

Lodash — Contoh Nyata

```
// ❌ BURUK — semua lodash masuk bundle (300 KB)
import _ from 'lodash';
_.debounce(fn, 300);
_.throttle(fn, 1000);
_.isEqual(a, b);

// ✅ BAIK — cuma fungsi yang dipake
import debounce from 'lodash/debounce';
import throttle from 'lodash/throttle';
import isEqual from 'lodash/isEqual';

// ✅ LEBIH BAIK — lo-dash tree-shakeable version
import { debounce, throttle, isEqual } from 'lodash-es';
```

---

3. Code Splitting Strategy

Split by Routes

```
// Setiap halaman jadi chunk sendiri
const Home = lazy(() => import('./pages/Home'));
const Product = lazy(() => import('./pages/Product'));
const Cart = lazy(() => import('./pages/Cart'));
const Checkout = lazy(() => import('./pages/Checkout'));
```

Split by Vendor

Pisahin library pihak ketiga biar ga ikut berubah tiap deploy:

```
// vite.config.ts
export default defineConfig({
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['react', 'react-dom'],
          utils: ['lodash-es', 'date-fns'],
          charts: ['recharts', 'd3'],
        },
      },
    },
  },
});
```

Split by Condition

```
// Cuma load kalo user adalah admin
async function loadAdminPanel() {
  if (user.role === 'admin') {
    const AdminPanel = await import('./AdminPanel');
    return new AdminPanel.default();
  }
}

// Cuma load kalo browser pake WebP
async function loadImageOptimizer() {
  const supportsWebP = document.createElement('canvas')
    .toDataURL('image/webp').startsWith('data:image/webp');

  const ImageModule = supportsWebP
    ? await import('./WebPOptimizer')
    : await import('./FallbackOptimizer');

  return new ImageModule.default();
}
```

---

4. Image Optimization

Format Comparison

Format	Kompresi	Kualitas	Browser Support
JPEG	Lossy	Baik	Semua
PNG	Lossless	Sangat baik	Semua
WebP	Lossy/Lossless	Setara JPEG 30% lebih kecil	96%
AVIF	Lossy/Lossless	Lebih kecil dari WebP 50%	80%
SVG	Vector	Tak terbatas	Semua
JPEG XL	Lossy/Lossless	Terbaik	Baru 10%

Sharp — Optimasi di Build Time

```
npm install --save-dev sharp
```

```
// scripts/optimize-images.ts
import sharp from 'sharp';
import fs from 'node:fs';
import path from 'node:path';
import { glob } from 'glob';
```

```
async function optimizeImages() {
  const images = await glob('public/images/**/*.{jpg,jpeg,png}');

  for (const file of images) {
    const ext = path.extname(file);
    const name = path.basename(file, ext);
    const dir = path.dirname(file);

    // WebP — kualitas 80
    await sharp(file)
      .webp({ quality: 80 })
      .toFile(`${dir}/${name}.webp`);

    // AVIF — kualitas 65
    await sharp(file)
      .avif({ quality: 65 })
      .toFile(`${dir}/${name}.avif`);

    // Resize buat thumbnail
    await sharp(file)
      .resize({ width: 400 })
      .webp({ quality: 60 })
      .toFile(`${dir}/${name}-thumb.webp`);

    console.log(`✓ ${name} → webp, avif, thumb`);
  }
}

optimizeImages().catch(console.error);
```

Picture Element — Browser Pilih Format

```
<picture>
  <source type="image/avif" srcset="photo.avif">
  <source type="image/webp" srcset="photo.webp">
  <img src="photo.jpg" width="800" height="600" alt="Photo" loading="lazy">
</picture>
```

Responsive Images

```
<img
  srcset="
    photo-400.webp 400w,
    photo-800.webp 800w,
```

```
    photo-1200.webp 1200w
  "
  sizes="
    (max-width: 600px) 400px,
    (max-width: 1200px) 800px,
    1200px
  "
  src="photo-800.webp"
  width="1200"
  height="800"
  loading="lazy"
  alt="Photo"
>
```

---

5. Font Loading Optimization

Masalah Font

Font besar (WOFF2 pun bisa 50-200 KB) blocking render. Tiga dampak:

1. **FOUT** (Flash of Unstyled Text) — font sistem dulu, baru font kustom
2. **FOIT** (Flash of Invisible Text) — teks ga kelihatan sampe font selesai
3. **CLS** — font kustom ukuran beda, layout geser

font-display

```css
/* @font-face dengan font-display */
@font-face {
  font-family: 'Inter';
  src: url('/fonts/Inter-Bold.woff2') format('woff2');
  font-weight: 700;
  font-display: swap; /* Tampilkan font sistem dulu, swap kalo selesai */
}

@font-face {
  font-family: 'Inter';
  src: url('/fonts/Inter-Regular.woff2') format('woff2');
  font-weight: 400;
  font-display: optional; /* Kalo font ga selesai dalam 100ms, pake font sistem
}
```

font-display	Perilaku	Cocok
auto	Browser decide (biasanya FOIT)	—
block	Invisible teks 3 detik	Brand font untuk logo
swap	Font sistem dulu, ganti kalo selesai	Body text
fallback	Swap cuma dalam 3 detik, abis itu stuck font sistem	Body text
optional	Kalo ga selesai 100ms, pake font sistem	Performance-critical

Preload Font

```
<link rel="preload" href="/fonts/Inter-Regular.woff2" as="font" type="font/woff2
<link rel="preload" href="/fonts/Inter-Bold.woff2" as="font" type="font/woff2" c
```

Subset Font — Kurangi Ukuran 90%

Hanya include karakter latin + angka + tanda baca (buang Cyrillic, Chinese, dll).

```
# Glyphhanger — generate subset font
npx glyphhanger ./index.html --subset=*.ttf --formats=woff2,woff --css

# Atau manual
npx glyphhanger https://example.com --subset=Inter-Regular.ttf --formats=woff2
```

Font Loading API

```
// Detect kalo font udah selesai load
document.fonts.ready.then(() => {
  console.log('Semua font selesai di-load');
  document.documentElement.classList.add('fonts-loaded');
});

/* CSS — sembunyikan FOUT sampe font siap */
html:not(.fonts-loaded) body {
  font-family: system-ui, sans-serif;
}

html.fonts-loaded body {
```

```
  font-family: 'Inter', system-ui, sans-serif;
}
```

---

6. Bundle Budget

Bundle budget = **batas maksimal ukuran file**. CI gagal kalo budget dilanggar.

```
// webpack.config.js — bundle budget checker
module.exports = {
  performance: {
    hints: 'error',     // Gagal build kalo budget dilanggar
    maxEntrypointSize: 250_000,    // 250 KB per entry
    maxAssetSize: 100_000,         // 100 KB per file
    assetFilter: (filename: string) =>
      !filename.endsWith('.map') && // Ga include sourcemap
      !filename.endsWith('.txt'),
  },
};
```

```
// vite.config.ts — pake plugin manual
import { visualizer } from 'rollup-plugin-visualizer';

export default defineConfig({
  build: {
    rollupOptions: {
      plugins: [
        visualizer({
          filename: 'dist/report.html',
          gzipSize: true,
        }),
      ],
    },
    chunkSizeWarningLimit: 100, // KB — warning kalo > 100 KB
  },
});
```

Target Budget

Resource	Max Size (gzipped)	Catatan
Total JS	≤ 300 KB	500 KB kalo pake framework
Total CSS	≤ 50 KB	—
Gambar (hero)	≤ 100 KB	WebP

Resource	Max Size (gzipped)	Catatan
Font	≤ 50 KB	WOFF2, subset
HTML	≤ 30 KB	—

Latihan

1. **Bundle Analyzer** — bikin React app baru (Vite). Tambahin `rollup-plugin-visualizer`. Build, buka report. Identifikasi 3 module terbesar. Terus refactor: ganti import besar dengan import spesifik. Bandingin size sebelum-sesudah
2. **Image Pipeline** — tulis script Node.js pake `sharp` yang:
   - Scan folder `public/images/`
   - Convert JPEG/PNG ke WebP (quality 75) + AVIF (quality 60)
   - Generate thumbnail 200px width buat tiap gambar
   - Output report: file asal → file baru + size reduction %
3. **Font Optimizer** — ambil font Google (Inter atau Poppins). Implementasi:
   - `font-display: swap` + `optional`
   - Preload WOFF2
   - Subset font (cuma latin)
   - Font loading API buat class `.fonts-loaded`
   - Ukur CLS sebelum-sesudah pake Lighthouse
4. **Bundle Budget CI** — setup bundle budget di project:
   - webpack: `performance: { hints: 'error', maxEntryPointSize: 250_000 }`
   - Vite: `chunkSizeWarningLimit: 100`
   - Test: tambahin library gede (moment.js / chart.js) — liat gagal build
   - Refactor pake alternatif ringan (date-fns / uPlot)

32.4 Lighthouse CI, Performance Budgets & Monitoring

---

1. Lighthouse Audit — Dasar

Lighthouse ngecek 5 kategori:

Kategori	Bobot	Yang Dinilai
Performance	50%	LCP, FID/INP, CLS, TBT, SI
Accessibility	20%	Contrast, aria labels, keyboard nav
Best Practices	15%	HTTPS, no errors, modern JS
SEO	10%	Meta tags, crawlability, robots.txt
PWA	5%	Service worker, manifest, offline

Run Lighthouse Manual

```
# CLI
npx lighthouse https://example.com --view

# Dengan Chrome DevTools Protocol
npx lighthouse https://example.com --output=json --output-path=./report.json

# Emulated mobile + throttling
npx lighthouse https://example.com \
  --preset=desktop \
  --throttling-method=devtools \
  --output=html \
  --output-path=./lighthouse-report.html
```

Lighthouse Programmatic API

```
import lighthouse from 'lighthouse';
import chromeLauncher from 'chrome-launcher';

async function runLighthouse(url: string) {
  const chrome = await chromeLauncher.launch({ chromeFlags: ['--headless'] });

  const options = {
    logLevel: 'info',
    output: 'json',
    port: chrome.port,
    onlyCategories: ['performance', 'accessibility', 'seo', 'best-practices'],
  };

  const config = {
    extends: 'lighthouse:default',
    settings: {
```

```
      formFactor: 'desktop',
      throttling: {
        rttMs: 40,
        throughputKbps: 10240,
        cpuSlowdownMultiplier: 1,
      },
    },
  };

  const result = await lighthouse(url, options, config);
  await chrome.kill();

  const scores = result?.lhr?.categories;
  if (!scores) return;

  console.log('Performance:', Math.round(scores.performance?.score ?? 0 * 100));
  console.log('Accessibility:', Math.round(scores.accessibility?.score ?? 0 * 10
  console.log('SEO:', Math.round(scores.seo?.score ?? 0 * 100));
  console.log('Best Practices:', Math.round(scores['best-practices']?.score ?? 0

  return result.lhr;
}
```

Interpretasi Skor

Skor	Warna	Arti
90–100	🟢 Hijau	Cepat & aksesibel
50–89	🟡 Kuning	Perlu perbaikan
0–49	🔴 Merah	Lambat, banyak masalah

---

2. CI/CD dengan Lighthouse CI

Setup Lighthouse CI (LHCI)

```
npm install --save-dev @lhci/cli
```

```
// lighthouserc.js
module.exports = {
  ci: {
    collect: {
      url: [
        'http://localhost:3000/',
```

```
        'http://localhost:3000/products',
        'http://localhost:3000/cart',
      ],
      startServerCommand: 'npm run start',
      numberOfRuns: 3,              // Rata-rata dari 3 run
      settings: {
        preset: 'desktop',
        onlyCategories: ['performance', 'accessibility', 'seo', 'best-practices'],
      },
    },
    upload: {
      target: 'filesystem',         // Simpan report lokal
      outputDir: './lhci-reports',
      reportFilenamePattern: 'lighthouse-%%URLNAME%%-%%DATETIME%%.html',
    },
    assert: {
      preset: 'lighthouse:no-pwa',  // Assertions default
      assertions: {
        'categories:performance': ['warn', { minScore: 0.9 }],
        'categories:accessibility': ['error', { minScore: 0.9 }],
        'categories:seo': ['error', { minScore: 0.9 }],
        'categories:best-practices': ['error', { minScore: 0.9 }],
        'lighthouse-disk-cache': 'off',
        'unused-javascript': ['warn', { maxLength: 0 }],
      },
    },
  },
};
```

GitHub Actions — Lighthouse CI

```
# .github/workflows/lighthouse.yml
name: Lighthouse CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```yaml
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Build
        run: npm run build

      - name: Run Lighthouse CI
        run: |
          npm install -g @lhci/cli
          lhci autorun
        env:
          LHCI_GITHUB_APP_TOKEN: ${{ secrets.LHCI_GITHUB_APP_TOKEN }}
```

GitLab CI — Lighthouse

```yaml
# .gitlab-ci.yml
image: node:20

stages:
  - build
  - test
  - lighthouse

lighthouse:
  stage: lighthouse
  script:
    - npm ci
    - npm run build
    - npm install -g @lhci/cli
    - lhci autorun --config=./lighthouserc.js
  artifacts:
    paths:
      - lhci-reports/
    expire_in: 30 days
  only:
    - main
```

---

3. Performance Budgets di CI

Lighthouse Assertions = Performance Budget

Assertions di `lighthouserc.js` otomatis ngegagalin CI kalo budget dilanggar:

```
// lighthouserc.js — assertions detail
assertions: {
  // Skor kategori
  'categories:performance': ['error', { minScore: 0.9 }],
  'categories:accessibility': ['error', { minScore: 0.9 }],

  // Metrik
  'largest-contentful-paint': ['error', { maxNumericValue: 2500 }],
  'cumulative-layout-shift': ['error', { maxNumericValue: 0.1 }],
  'total-blocking-time': ['error', { maxNumericValue: 200 }],
  'interactive': ['error', { maxNumericValue: 5000 }],
  'speed-index': ['warn', { maxNumericValue: 4000 }],

  // Resource size
  'total-byte-weight': ['error', { maxNumericValue: 1_500_000 }], // 1.5 MB
  'unused-javascript': ['warn', { maxLength: 0 }],
  'unused-css-rules': ['warn', { maxLength: 0 }],

  // Best practices
  'uses-http2': ['error', { minScore: 1 }],
  'uses-long-cache-ttl': ['error', { maxLength: 0 }],
  'uses-responsive-images': ['error', { maxLength: 0 }],
}
```

Danger.js — Post PR Comment

```
// dangerfile.ts
import { danger, fail, warn, markdown } from 'danger';
import fs from 'node:fs';

const lhciReport = JSON.parse(
  fs.readFileSync('./lhci-reports/manifest.json', 'utf-8')
);

for (const url of Object.keys(lhciReport)) {
  const report = lhciReport[url];
  const performance = Math.round(report.summary.performance * 100);
  const accessibility = Math.round(report.summary.accessibility * 100);

  if (performance < 90) {
```

```
    fail(`□ Performance skor ${performance} di ${url} — target minimal 90`);
  }

  if (accessibility < 90) {
    warn(`⚠ Accessibility skor ${accessibility} di ${url} — target minimal 90`);
  }
}

markdown(`
## □ Lighthouse Report
| URL | Performance | Accessibility |
|-----|-------------|---------------|
${Object.entries(lhciReport)
  .map(([url, r]) => `| ${url} | ${Math.round(r.summary.performance * 100)} | ${
  .join('\n')}
`);
```

---

4. Core Web Vitals Monitoring (web-vitals Library)

Setup web-vitals

```
npm install web-vitals
```

```
// src/metrics.ts
import { onLCP, onFID, onCLS, onINP, onTTFB } from 'web-vitals';

interface MetricReport {
  name: string;
  value: number;
  rating: 'good' | 'needs-improvement' | 'poor';
  delta: number;
  id: string;
}

function reportToAnalytics(metric: MetricReport) {
  console.log(`[Web Vitals] ${metric.name}: ${metric.value.toFixed(2)} (${metric

  // Kirim ke endpoint
  const body = JSON.stringify({
    metric: metric.name,
    value: metric.value,
    rating: metric.rating,
    url: window.location.pathname,
    timestamp: Date.now(),
  });
```

```
  if (navigator.sendBeacon) {
    navigator.sendBeacon('/api/metrics', body);
  } else {
    fetch('/api/metrics', { method: 'POST', body, keepalive: true });
  }
}

// Daftarin semua observers
export function initWebVitals() {
  onLCP(reportToAnalytics);
  onFID(reportToAnalytics);
  onCLS(reportToAnalytics);
  onINP(reportToAnalytics);
  onTTFB((metric) => {
    console.log(`TTFB: ${metric.value.toFixed(0)}ms`);
  });
}
```

```
// main.ts — panggil di entry point
import { initWebVitals } from './metrics';

initWebVitals();
```

web-vitals dengan React

```
// src/hooks/useWebVitals.ts
import { useEffect } from 'react';
import { onLCP, onFID, onCLS, onINP } from 'web-vitals';

export function useWebVitals() {
  useEffect(() => {
    const report = (metric: any) => {
      console.log(`[${metric.name}] ${metric.value.toFixed(2)} — ${metric.rating
    };

    onLCP(report);
    onFID(report);
    onCLS(report);
    onINP(report);
  }, []);
}
```

Dashboard Visual — buat grafik metrik

```
// Ambil data dari endpoint
async function fetchMetrics(timeRange = '24h') {
  const res = await fetch(`/api/metrics?range=${timeRange}`);
  const data = await res.json();

  // Kelompokin per metrik
  const grouped = data.reduce((acc: any, item: any) => {
    if (!acc[item.metric]) acc[item.metric] = [];
    acc[item.metric].push(item);
    return acc;
  }, {});

  console.table(grouped);

  // Hitung persentase good / needs-improvement / poor
  for (const [metric, entries] of Object.entries(grouped)) {
    const total = (entries as any[]).length;
    const good = (entries as any[]).filter((e: any) => e.rating === 'good').leng
    const poor = (entries as any[]).filter((e: any) => e.rating === 'poor').leng

    console.log(`${metric}: ${((good / total) * 100).toFixed(1)}% good, ${((poor
  }
}
```

---

5. Common Pitfalls & Fixes

Masalah	Penyebab	Fix
Skor Performance rendah	JS terlalu besar, gambar gede	Code splitting, image optimization
LCP merah	Hero image lambat, server slow	Preload hero, CDN, SSG
CLS tinggi	Gambar/font tanpa dimensi	`width`/`height`, `aspect-ratio` CSS
TBT (Total Blocking Time)	Long task di main thread	Web Worker, chunk task

Masalah	Penyebab	Fix
Accessibility rendah	Kurang aria labels, contrast rendah	Gunakan semantic HTML, alat otomatis
SEO rendah	Meta tag kurang, no sitemap	Tambah meta description, canonical
Unused JavaScript	Import library gede tapi jarang dipake	Dynamic import, tree shaking
Render-blocking resources	CSS/JS di <head> blocking	Inline critical CSS, `defer`/async
Image not optimized	PNG/JPEG tanpa kompresi	Sharp pipeline ke WebP/AVIF
No HTTP/2	Server belum pake HTTP/2	Enable di nginx/caddy
Cache policy buruk	Ga pake Cache-Control	Set `max-age` sesuai jenis resource
No service worker	Ga bisa offline & instant load	Register SW + cache strategi

Checklist Cepat Optimasi

```
- [ ] Core Web Vitals ≥ target (LCP ≤ 2.5s, CLS ≤ 0.1, INP ≤ 200ms)
- [ ] Gambar pake WebP/AVIF + loading="lazy"
- [ ] Hero image preload + fetchpriority="high"
- [ ] Code splitting per route / per komponen berat
- [ ] Font: WOFF2 + font-display + preload
- [ ] Cache-Control: statis immutable 1 tahun, API no-cache
- [ ] Service worker dengan strategi caching
- [ ] Bundle size: JS ≤ 300 KB gzip, CSS ≤ 50 KB
- [ ] Lighthouse score ≥ 90 semua kategori
- [ ] Lighthouse CI di pipeline
- [ ] web-vitals monitoring di production
- [ ] HTTP/2 + CDN
```

Latihan

1. **Lighthouse Runner** — tulis script Node.js yang:
   - Jalankan Lighthouse di 3 URL (home, produk, checkout)
   - Ekstrak skor performance + LCP + CLS
   - Output ke console.table()
   - Simpan ke JSON file
2. **LHCI Config** — bikin `lighthouserc.js` lengkap:
   - 3 URL target
   - Assertions: perf ≥ 0.9, LCP ≤ 2500, CLS ≤ 0.1, TBT ≤ 200
   - Upload target filesystem
   - Jalanin `lhci autorun` — liat hasil pass/fail
3. **web-vitals Dashboard** — bikin halaman HTML yang:
   - Pake `web-vitals` library
   - Kirim metrik ke localStorage (simulasi analytics)
   - Tampilin tabel real-time: nama metrik | value | rating
   - Tambahin warna: hijau/kuning/merah sesuai threshold
4. **CI Pipeline** — bikin GitHub Actions workflow `.github/workflows/perf.yml`:
   - Trigger: push ke main, PR ke main
   - Step: checkout → setup node → install → build → lhci autorun
   - Assertions: perf ≥ 0.9, aksesibilitas ≥ 0.9
   - Upload report sebagai artifact
   - Tambahin badge status ke README